

Platform Engineering Design Exercise

AWS Infrastructure Provisioning Platform using Terraform / Terragrunt, and GitHub Actions

Objective

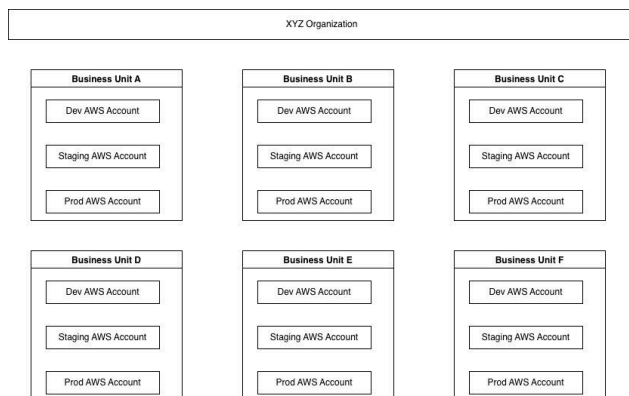
Design an enterprise-scale Infrastructure as Code (IaC) platform that enables DevOps engineers and application teams to provision and manage AWS infrastructure consistently across multiple Business Units (BUs) using Terraform / Terragrunt, and GitHub Actions.

The platform should support:

- Multi-account AWS architecture
 - Shared-nothing isolation between Business Units
 - Self-service infrastructure provisioning
 - Reusable infrastructure modules
 - Standardized CI/CD workflows
 - Secure authentication and secret management
 - Controlled emergency changes and drift management
 - Enterprise-grade governance and scalability
-

Existing AWS Organization Structure

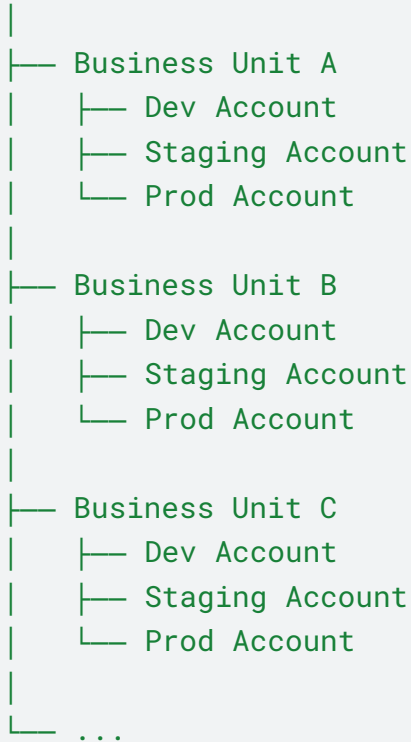
The organization contains multiple Business Units.



Each Business Unit has three AWS accounts:

None

XYZ Organization



Assumptions

- One environment = one AWS account
- Shared nothing architecture
- Infrastructure pattern is largely identical across environments
- Business Units may have additional application-specific infrastructure
- Platform should scale to 50+ Business Units

Problem Statement

Design an AWS Infrastructure Provisioning Platform.

You are expected to describe:

1. Repository strategy
2. Terraform/Terragrunt structure
3. Module strategy

4. Versioning approach
 5. State management
 6. Secret management
 7. Deployment workflow
 8. AWS authentication model
 9. Account provisioning strategy
 10. Developer self-service model
 11. Emergency change process
 12. Drift detection and remediation
 13. Governance and security controls
-

Design Topics

1. Repository Strategy

Describe:

Option A

Single monorepo

Option B

Multiple repositories

Option C

Hybrid approach

Explain:

- Pros and cons
- Scalability considerations
- Team ownership model
- Release management model

Provide your recommended approach.

2. Repository Structure

Show the GitHub repository layout you would propose.

Explain:

- Purpose of each repository
 - Ownership model
 - Promotion process
-

3. Terraform vs Terragrunt

Explain:

- Why Terragrunt is needed
- How DRY principles are achieved
- How account-specific configuration is managed
- How environment-specific variables are managed

Provide folder structure examples.

4. Reusable Module Strategy

Describe:

Common Infrastructure

Examples:

- VPC
- Transit Gateway
- Route53
- CloudWatch
- GuardDuty
- Security Hub
- IAM baseline

Application Infrastructure

Examples:

- ECS
- Lambda
- API Gateway
- SQS
- SNS
- RDS
- DynamoDB
- S3

Questions:

- Build custom modules?
- Use official Terraform modules?
- Fork community modules?
- Wrapper modules?

Explain your approach.

5. Module Versioning Strategy

Describe:

- Semantic versioning approach
- Backward compatibility
- Breaking change management
- Module upgrade process

How will Business Units consume new versions?

6. DRY Across Business Units

All Business Units require:

- Networking
- Logging
- Monitoring
- Security baseline
- IAM baseline

How would you avoid duplicating Terraform code?

Explain:

- Terragrunt inheritance
 - Shared configuration
 - Reusable modules
 - Environment overlays
-

7. DRY Across Environments

Within a Business Unit:

```
None
BU-A
├─ Dev
├─ Staging
└─ Prod
```

Infrastructure is mostly identical.

Explain:

- How to avoid duplication
 - How to handle environment-specific settings
 - How variables are managed
-

8. State Management

Design Terraform state management.

Explain:

Backend

- S3
- DynamoDB locking (or alternative)

State Isolation

Questions:

- One state file per environment?
 - One state file per component?
 - State promotion strategy?
-

9. Secret Management

Explain:

GitHub Secrets

AWS Secrets Manager

Parameter Store

Vault

Questions:

- Where should secrets live?
 - How are secrets injected?
 - How are secrets rotated?
-

10. GitHub Actions Design

Design deployment workflows.

Explain:

Reusable workflows

Questions:

- How many workflows?
 - How are workflows shared?
 - Approval model?
 - Promotion model?
-

11. AWS Authentication

GitHub Actions runners need access to AWS accounts.

Explain:

OIDC Federation

IAM Roles

Role Trust Policies

Questions:

- How does authentication work?
 - How are permissions scoped?
 - How do you separate Dev/Staging/Prod access?
-

12. AWS Account Provisioning

New Business Unit onboarding.

Describe:

- How accounts are created
- AWS Organizations usage
- Account Factory
- Control Tower (optional)

Explain onboarding workflow.

13. Deployment Strategy

Explain:

Pull Request Flow

Questions:

- How are plans reviewed?
 - Who approves?
 - How are production deployments controlled?
-

14. Emergency Changes

Production outage scenario.

Questions:

- Can engineers change AWS manually?
- How is emergency access granted?
- How is the change tracked?
- How is Terraform state reconciled later?

Provide an operational process.

15. Drift Detection

Manual changes may introduce drift.

Explain:

- Detection strategy
 - Automated drift scans
 - Reporting
 - Remediation workflow
-

16. Developer Self-Service

The platform will be used by:

- Platform Engineers
- DevOps Engineers
- Software Engineers

Application teams should be able to provision infrastructure without deep Terraform knowledge.

Examples:

None

ECS Service

Lambda

S3 Bucket

SQS Queue

RDS Database

Explain:

- How complexity is abstracted
- Self-service workflow
- Templates / blueprints
- Golden paths
- Catalog approach

Provide examples of how a developer would request infrastructure.

Deliverables Expected

Please prepare:

1. Architecture Diagram

High-level platform architecture.

2. Repository Structure

GitHub repository strategy and folder layout.

3. Terraform and Terragrunt Design

Configuration hierarchy and DRY strategy.

4. Deployment Workflow

End-to-end deployment process.

5. Security Model

Authentication, authorization, secrets, and governance.

6. Operational Model

- Module lifecycle
- Version management
- Emergency changes

- Drift remediation
-

7. Self-Service Model

How application teams consume the platform.

Evaluation Criteria

We are evaluating:

Area	Focus
Terraform	Module design, state management, versioning
Terragrunt	DRY architecture, inheritance strategy
AWS	Multi-account platform design
GitHub Actions	CI/CD architecture and reusable workflows
Security	OIDC, IAM, secrets, governance
Scalability	Ability to support many Business Units
Operability	Day-2 operations and supportability
Developer Experience	Self-service and abstraction
Decision Making	Trade-offs and rationale

Submission Guidelines

Please submit your solution in the following formats.

GitHub Repository

Example structure:

```
None
README.md

/docs

  architecture.md

  deployment-workflow.md

  security-model.md

  operational-model.md

/diagrams

  platform-architecture.png

  repository-structure.png

/examples

  terragrunt-layout.md

  repository-layout.md
```

The repository does not need to contain production-ready code.

Pseudocode, diagrams, examples, and design documentation are sufficient.

Time Expectation

Please prepare a solution suitable for a 30 minute walkthrough.

The objective is to understand how you think about platform engineering, infrastructure architecture, scalability, security, governance, and developer experience.

We are not evaluating Terraform syntax memorization. We are evaluating architecture decisions, trade-offs, and operational thinking.